

WEEK 5: Payment Transaction Management System

- ATCHAYA S [E25AI106]

Scenario

The phone case business allows customers to make payments for their orders using different payment methods, such as UPI, Credit Card, Debit Card and Cash. The system stores payment details, including Payment ID, Order ID, Payment Date, Payment Mode, Payment Status and Transaction Amount.

The system is used to manage payment transactions, track successful, failed and pending payments, update payment status after retries and generate useful payment analysis reports.

1. Create Payment Table

```
SQL> CREATE TABLE Payment (  
 2     Payment_ID NUMBER PRIMARY KEY,  
 3     Order_ID NUMBER NOT NULL,  
 4     Payment_Date DATE NOT NULL,  
 5     Payment_Mode VARCHAR2(20) NOT NULL,  
 6     Payment_Status VARCHAR2(20) NOT NULL,  
 7     Transaction_Amount NUMBER(10,2) NOT NULL,  
 8     CONSTRAINT fk_payment_order  
 9         FOREIGN KEY (Order_ID)  
10         REFERENCES Orders(Order_ID),  
11     CONSTRAINT chk_payment_amount  
12         CHECK (Transaction_Amount > 0)  
13 );
```

Table created.

```
SQL> desc Payment;
```

Name	Null?	Type
PAYMENT_ID	NOT NULL	NUMBER
ORDER_ID	NOT NULL	NUMBER
PAYMENT_DATE	NOT NULL	DATE
PAYMENT_MODE	NOT NULL	VARCHAR2(20)
PAYMENT_STATUS	NOT NULL	VARCHAR2(20)
TRANSACTION_AMOUNT	NOT NULL	NUMBER(10,2)

2. Inserting Values

```
SQL> INSERT INTO Payment VALUES
  2  (1, 1001, '20-SEP-26', 'UPI', 'Completed', 1299);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2  (2, 1002, '21-SEP-26', 'Credit Card', 'Completed', 899);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2  (3, 1003, '22-SEP-26', 'Debit Card', 'Completed', 1599);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2  (4, 1004, '23-SEP-26', 'UPI', 'Pending', 699);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2  (5, 1005, '24-SEP-26', 'Credit Card', 'Completed', 450);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2  (6, 1006, '24-SEP-26', 'Cash', 'Pending', 750);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2  (7, 1007, '24-SEP-26', 'UPI', 'Completed', 1200);

1 row created.
```

3. Viewing Values in Table

```
SELECT * FROM Payment;
```

TRANSACTION_AMOUNT				
4	1004	23-SEP-26	UPI	Pending
	699			
5	1005	24-SEP-26	Credit Card	Completed
	450			
6	1006	24-SEP-26	Cash	Pending
	750			
PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
TRANSACTION_AMOUNT				
7	1007	24-SEP-26	UPI	Completed
	1200			
8	1008	25-SEP-26	Debit Card	Completed
	550			
9	1009	26-SEP-26	UPI	Pending
	999			
PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
TRANSACTION_AMOUNT				
11	1011	24-SEP-26	UPI	Completed
	450			
10	1010	24-SEP-26	Credit Card	Completed
	550			
11 rows selected.				

4. Establish Relationships

Orders and Payment

- One order can have a payment transaction.
- Each payment transaction is associated with one order.
- The Order_ID in the Payment table is used as a foreign key referencing the Orders table.

Relationship:

Orders (1) → Payment (Many)

This relationship allows the system to associate payment transactions with their respective orders.

5. Apply Constraints

The following constraints are applied while creating the table:

- **Primary Key:** Payment_ID uniquely identifies each payment transaction.
- **Foreign Key:** Order_ID references Orders(Order_ID).
- **NOT NULL:** Essential fields such as Order ID, Payment Date, Payment Mode, Payment Status and Transaction Amount cannot be left empty.
- **CHECK Constraint:** Payment status is restricted to Completed, Failed Or Pending.
- **CHECK Constraint:** Transaction amount must be greater than zero.

6. Update and Delete Records

Update payment status after retry

```
SQL> UPDATE Payment
  2  SET Payment_Status = 'Pending'
  3  WHERE Payment_ID = 10;

1 row updated.

SQL> SELECT * FROM Payment
  2  WHERE Payment_ID = 10;

PAYMENT_ID  ORDER_ID  PAYMENT_D  PAYMENT_MODE  PAYMENT_STATUS
-----
TRANSACTION_AMOUNT
-----
          10         1010 24-SEP-26 Credit Card          Pending
                    550

SQL> DELETE FROM Payment
  2  WHERE Payment_ID = 10;

1 row deleted.
```

7. Payment Transaction Queries

REPORT 1. Display All Successful Payments

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Completed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
1	1001	20-SEP-26	UPI	Completed
2	1002	21-SEP-26	Credit Card	Completed
3	1003	22-SEP-26	Debit Card	Completed

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
5	1005	24-SEP-26	Credit Card	Completed
7	1007	24-SEP-26	UPI	Completed
8	1008	25-SEP-26	Debit Card	Completed

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
11	1011	24-SEP-26	UPI	Completed

7 rows selected.

This query displays all successfully completed payment transactions.

REPORT 2. Find Failed Transactions

```
SQL> SELECT *
2 FROM Payment
3 WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
10	1010	24-SEP-26	Credit Card	Failed
12	1012	24-SEP-26	Credit Card	Failed

This query identifies all failed payment transactions that may require a retry.

REPORT 3. Count Total Successful and Failed Payments

```
SQL> SELECT Payment_Status, COUNT(*) AS Total_Payments
2 FROM Payment
3 WHERE Payment_Status IN ('Completed', 'Failed')
4 GROUP BY Payment_Status;
```

PAYMENT_STATUS	TOTAL_PAYMENTS
Failed	2
Completed	7

This query calculates the total number of successful and failed payment transactions.

REPORT 4. Update Failed Payments After Retry

```
SQL> UPDATE Payment
2 SET Payment_Status = 'Completed'
3 WHERE Payment_ID = 12
4 AND Payment_Status = 'Failed';

1 row updated.
```

```
SQL>
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_ID = 12;
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
12	1012	24-SEP-26	Credit Card	Completed
	550			

This query updates the payment status to Completed when a failed transaction is successfully processed after retry.

REPORT 5. Identify Pending Transactions

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Pending';
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
4	1004	23-SEP-26	UPI	Pending
	699			
6	1006	24-SEP-26	Cash	Pending
	750			
9	1009	26-SEP-26	UPI	Pending
	999			

This query identifies all pending payment transactions that are awaiting confirmation or further processing.

Conclusion

The Payment Transaction Management System helps manage and monitor customer payments. It supports payment record creation, status tracking and updating failed transactions after retries. The five SQL queries help identify successful, failed and pending transactions and provide payment counts for transaction analysis.

